

On the tour planning problem

Chenbo Zhu · J.Q. Hu · Fengchun Wang · Yifan Xu ·
Rongzeng Cao

Published online: 12 June 2010
© Springer Science+Business Media, LLC 2010

Abstract Increasingly, tourists are planning trips by themselves using the vast amount of information available on the Web. However, they still expect and want trip plan advisory services. In this paper, we study the tour planning problem in which our goal is to design a tour trip with the most desirable sites, subject to various budget and time constraints. We first establish a framework for this problem, and then formulate it as a mixed integer linear programming problem. However, except when the size of the problem is small, say, with less than 20–30 sites, it is computationally infeasible to solve the mixed-integer linear programming problem. Therefore, we propose a heuristic method based on local search ideas. The method is efficient and provides good approximation solutions. Numerical results are provided to validate the method. We also apply our method to the team orienteering problem, a special case of the tour planning problem which has been considered in the literature, and compare our method with other existing methods. Our numerical results show that our method produces very good approximation solutions with relatively small computational efforts comparing with other existing methods.

Keywords Tour planning problem · Team orienteering problem · Mixed integer linear programming · Heuristic · Local search

This work is supported in part by a grant from IBM China Research Laboratory, by the National Natural Science Foundation of China under Grant NSFC No. 70832002 and 70771028, and by the Shanghai Science and Technology PuJiang Funds under Grant 09PJ1401500. The authors would like to thank two anonymous reviewers for their comments and suggestions.

C. Zhu · J.Q. Hu (✉) · Y. Xu

Department of Management Science, School of Management, Fudan University, Shanghai 200433, China

e-mail: hujq@fudan.edu.cn

F. Wang · R. Cao

IBM China Research Laboratory, Beijing 100193, China

1 Introduction

According to World Travel & Tourism Council, Travel & Tourism is one of the world's highest priority industries and employers. It encompasses transportation, accommodation, catering, recreation and services for visitors both at home and from overseas. Information communication and technologies have radically changed the efficiency and effectiveness of tourism organizations, the way that businesses catering to tourists operate, as well as how consumers interact with organizations (Buhalis 2003). One of the major paradigm shifts is due to the emergence of the Internet. A survey of US Web users conducted in 2007 by Burstmedia shows that four out of five respondents would use the Internet to plan their upcoming personal travels.

Generally speaking, most travel websites provide information, destination or travel package recommendation, on-line flight/hotel/car reservation, and community forums for sharing travel tips. With the help of such travel websites, a new type of user is emerging: he becomes his own travel agent and builds his travel packages himself (Hannes 2003). Trip planning is a complex constructive activity affected by various factors, which can be classified into two categories: (1) personal features, including both socioeconomic factors (such as age, education and income) and psychological and cognitive factors (such as experience, personality, and involvement), and (2) travel features (such as travel purpose, travel-party size, length of travel, distance, and transportation mode). In the study of the complex constructive activity, both researchers and practitioners have explored several possible approaches. One popular approach is to use recommendation systems based on Artificial Intelligence (Buhalis and Law 2008). In a travel recommender system (TRS), travelers' needs and constraints, through recommendation algorithms, are mapped into appropriate product selections based on the knowledge collected by the intelligent recommender. Knowledge can be extracted by using the following four approaches (Schafer et al. 1999):

1. Non-personalized: recommending products to customers based on what other customers have said about the products on average;
2. Attribute based: recommending products to customers based on syntactic properties of the products;
3. Item-to-item correlations: recommending products to customers based on a small set of products the customers have expressed interest in;
4. People-to-people correlations: recommending products to a customer based on the correlation between that customer and other customers who have purchased products from the E-commerce site.

However, Ricci (2002) argues that none of these approaches can support the user in building a "user defined" trip package that consists of one or more tourist attractions, accommodations, catering and other services. Current TRS can only support the first stage of trip package planning-destination selection, because: (1) a content based approach does not scale unless one pursues a costly knowledge-engineering activity, and (2) a collaborative-filtering approach encounters the complexity of the travel products: one cannot simplify a trip to the point where two travelers' trips are the same. Fesenmainer et al. (2003) propose several approaches to help a tourist to design a personalized tour plan. In the first approach it offers the tourist a list of items based on his preference, the tourist can then design a personalized tour that combines several items in the list. In the second approach, it provides the tourist a set of pre-packaged tour plans based on his preference from which the tourist can choose one. In the third approach, it provides the tourist with various tour plans previously used by other tourists from which the tourist can make a choice. Ricci and Wether (2002) believe

that a case based reasoning (CBR) approach might be more useful. The recommendation process of the CBR approach works as follows: (1) the tourist gives his preference about an item, say, a hotel; (2) the recommendation system searches the catalogue and retrieves some hotels satisfying the tourist's preference; (3) previous stored tour plans are used to rank these hotels, and the hotel that the same tourist or similar tourists have used in these tour plans is ranked first; (4) the tourist selects his hotel from this ranked list.

No matter what kind of recommendation systems are employed as advisors for tour package planning, a reference base would be extremely useful and can be used to simplify the selection process dramatically: the reference base could be either previous purchase/selection records by the user/other users or a prepackaged tour provided by a travel agency or other experienced travelers. Although some commercial websites claim that they can support tour trip planning, it's more like an electronic notebook and organizer, which allows the user to record his selections of single products (e.g., a flight, a hotel, a car rental) in his trip planning. The user still has to plan his entire trip almost completely manually with little help from these websites.

In this paper, we establish a framework for the tour planning problem (TPP), and then develop a mathematical model as well as efficient algorithms. In solving such a problem, our objective is to design a tour trip with the most desirable sites subject to various budget and time constraints. We first formulate the problem as a mixed integer linear programming (MILP) problem. However, as in many similar applications, the resulting MILP problem is NP-complete and it is computationally difficult to solve when the problem size is large. Therefore, we propose a heuristic method based on local search ideas. We also provide numerical results to show that the method is efficient and provides good approximation solutions.

A related problem to TPP is the team orienteering problem (TOP) (Chao et al. 1996), which has been studied in the literature. TOP is a special case of our tour planning problem in which the visiting time to each site is zero and both the starting and end points of the tour are fixed and given. Recently, various algorithms have been proposed for TOP, e.g., Archetti et al. (2007), Ke et al. (2008), Vansteenwegen et al. (2009). Archetti et al. (2007) develop two tabu search variants and a variable neighborhood search algorithm. Ke et al. (2008) propose four different methods to construct candidate solutions in the framework of the ant colony optimization approach. Vansteenwegen et al. (2009) propose two local search heuristics, and apply guided local search method to improve them. All these algorithms are based on heuristics. The algorithm that we develop in this paper can be applied to TPP, a broader class of problems which include TOP as a special case. In addition, our algorithm is very simple and easy to implement. Our numerical results also show that when applied to TOP, our algorithm provides very good approximation solutions with relatively small computational efforts comparing with other existing algorithms in the literature.

The rest of the paper is organized as follows. In Sect. 2, we first introduce TPP and formulate the problem as a mixed-integer linear programming problem. In Sect. 3, we present our algorithm, with numerical examples provided in Sect. 4 to validate the algorithm. In Sect. 5, we discuss TOP, a special case of TPP. We apply our algorithm to TOP and compare it with the existing algorithms. Finally, a conclusion and some discussions are provided in Sect. 6.

2 Problem formulation

Let $G = (V, E)$ be a connected graph with vertices $V = M \cup N$, where $M = \{1, \dots, m\}$ represents m hotel sites and $N = \{m + 1, \dots, m + n\}$ represents n tourist sites, and edges

$E = \{(i, j) : i, j \in V\}$. For ease of exposition, we assume that only one hotel is selected for the entire tour. However, our formulation and method can be extended to more general cases in which more than one hotel may be selected. We define:

- C the total budget available for the tour,
- d the number of days available for the tour, and let $D = \{1, \dots, d\}$,
- T_k the tour time available during the k th day ($k \in D$),
- τ_i the tour time that the tourist spends at site i , $i \in V$, ($\tau_i = 0$ for $i \in M$),
- (a_i, b_i) the time period during which the tourist is allowed to visit site i , $i \in V$, and we assume that it is the same every day,
- c_i the cost associated with visiting (or staying at) site i , $i \in V$,
- e_{ij} the cost associated with traveling between site i and j , $i, j \in V$,
- U_i the utility of visiting (or staying at) site i , $i \in V$,
- t_{ij} the travel time between sites i and j , $i, j \in V$.

We note that one way to obtain the utility function for each site is to introduce a set of attributes, A (e.g., an attribute could be natural beauty, historical significance, or cultural heritage). Each attribute is assigned a weight w_i by the tourist, which indicates his preference to the attribute (the higher the weight, the more preferred the attribute by the tourist). Let $S = \{s_{ij} : i \in V, j \in A\}$ be the attribute value set, where s_{ij} is the value of attribute j for site i . For example, if site i is a historical museum, then its value of the historical attribute is very high, while its values for other attributes may be very small (or zero). Given S and W , we can obtain tourist's utility U_i for site i as: $U_i = \sum_{j \in A} w_j s_{ij}$.

The tour planning problem (TPP) is to construct a d -day tour that maximizes the total utility of all sites visited while satisfying the following constraints:

- The number of days for the tour is d ;
- For each day, the tour starts and ends at the hotel;
- Each tourist site is visited no more than once;
- The arrival and the departure time at each tourist site is restricted by its permitted visiting time $[a_i, b_i]$;
- The cumulative tour time of the k th day does not exceed the available tour time T_k ;
- The total cost of the d -day tour does not exceed the budget C .

We now introduce the following decision variables:

$$x_{ijk} = \begin{cases} 1, & \text{if the tourist travels from site } i \text{ to site } j \text{ on the } k\text{th day,} \\ 0, & \text{otherwise.} \end{cases}$$

$$y_i = \begin{cases} 1, & \text{if site } i \text{ is visited by the tourist,} \\ 0, & \text{otherwise.} \end{cases}$$

$$z_i = \begin{cases} 1, & \text{if hotel } i \text{ is selected by the tourist,} \\ 0, & \text{otherwise.} \end{cases}$$

t_i = the time epoch at which the tourist starts his visit at site i ,

t_{ik}^a = the time epoch at which the tourist departs from hotel i on the k th day,

t_{ik}^b = the time epoch at which the tourist arrives at hotel i on the k th day.

TPP can then be formulated as the following MILP problem:

$$\max \quad \sum_{i \in N} U_i y_i + \sum_{i \in M} U_i z_i \quad (1)$$

$$\text{s.t.} \quad \sum_{i \in V} x_{irk} = \sum_{j \in V} x_{rjk}, \quad r \in N, k \in D, \quad (2)$$

$$\sum_{i \in N} x_{irk} = \sum_{j \in N} x_{rjk}, \quad r \in M, k \in D, \quad (3)$$

$$y_i = \sum_{k \in D} \sum_{j \in V} x_{ijk}, \quad i \in N, \quad (4)$$

$$z_i = \frac{1}{d} \sum_{k \in D} \sum_{j \in N} x_{ijk}, \quad i \in M, \quad (5)$$

$$\sum_{i \in M} z_i = 1 \quad (6)$$

$$t_i + \tau_i + t_{ij} - (1 - x_{ijk})T_k \leq t_j, \quad i, j \in N, k \in D, \quad (7)$$

$$t_{ik}^a + t_{ij} - (1 - x_{ijk})T_k \leq t_j, \quad i \in M, j \in N, k \in D, \quad (8)$$

$$t_i + \tau_i + t_{ij} - (1 - x_{ijk})T_k \leq t_{jk}^b, \quad i \in N, j \in M, k \in D, \quad (9)$$

$$a_i \leq t_i \leq b_i, \quad i \in N, \quad (10)$$

$$a_k \leq t_{ik}^a \leq t_{ik}^b \leq b_k, \quad i \in M, k \in D, \quad (11)$$

$$\sum_{i \in N} \left(c_i y_i + \sum_{k \in D} \sum_{j \in V} e_{ij} x_{ijk} \right) + \sum_{i \in M} \left(d c_i z_i + \sum_{k \in D} \sum_{j \in N} e_{ij} x_{ijk} \right) \leq C \quad (12)$$

$$x_{ijk} \in \{0, 1\}, \quad i, j \in V, k \in D, \quad (13)$$

$$y_i \in \{0, 1\}, \quad i \in N, \quad (14)$$

$$z_i \in \{0, 1\}, \quad i \in M. \quad (15)$$

In the above MILP, the objective function (1) maximizes the total utility of all sites visited by the tourist and the constraints are:

- Constraint (2) is the balance equation, which ensures that if the tourist visits a site, he would leave the site;
- Constraint (3) ensures that the tourist starts and ends at the same hotel every day;
- Constraints (4) and (5) state the relationship between x_{ijk} , y_i , and z_i ;
- Constraint (6) ensures that only one hotel is selected for the entire tour;
- Constraints (7)–(11) are the time constraints;
- Constraint (12) is the budget constraint.

We point out that the MILP can be decomposed into smaller one-hotel sub-problems. It is clear that each sub-problem is much easier to solve compared with the original problem. However, it still remains difficult to solve computationally.

We note that TPP is related to the classical Vehicle Routing Problem (VRP). In VRP, the objective is to design the routes for a fleet of vehicles to service all customers with minimum total cost, subject to vehicle capacity and time constraints. The cost in VRP can

be either the number of vehicles used or the total length of the routes. In graphical theory terminology, VRP is to find a set of cycles with minimum total cost to cover all the vertices. By comparison, TPP is to generate a fixed number of cycles such that the total utility value of the vertices covered by these cycles is maximized.

3 Our algorithm

In this section we present our heuristic algorithm based on local search ideas. First, we outline the basic structure of the algorithm, and then present the key components of the algorithm in detail.

3.1 Basic structure

Our algorithm includes two major procedures: (1) hotel selection, and (2) tour planning. In Procedure 1, the hotels are first divided into R groups according to their locations. We may set an upper limit on the number of hotels in each group. So if one group contains too many hotels, we could divide them further into smaller groups, until the number of hotels in each group does not exceed the upper limit. In our numerical experiments, for example, the hotels are divided based on their district locations (regions). We then randomly select one hotel from each group and apply Procedure 2 to this selected hotel and obtain “BestSolution” for the visiting tour associated with this hotel. We next choose the group with “BestSolution”, say, group r . Finally, we apply Procedure 2 to every hotel in group r , and select the hotel in group r with “BestSolution”, which is used as our final solution. The details of Procedure 1 are provided in the following.

Procedure 1 Hotel selection

```

While CurrentSearchGroup contains too many hotels do
  Divide CurrentSearchGroup into  $R$  groups;
  For Region  $r = 1$  to  $R$ 
    Randomly selected a hotel  $i \in$  group  $r$  and apply Procedure 2 to hotel  $i$ ;
  End
  Select the group with the best solution as CurrentSearchGroup;
End
For For every hotel  $i \in$  CurrentSearchGroup
  Apply Procedure 2 to hotel  $i$ ;
End
Return BestSolution.

```

In Procedure 2, we start with an initial solution (obtained from Initialization routine), and then apply two main procedures: time reduction (with 2-opt*, Move, 2-opt, and Or-opt routines) and utility increase (with Insert and Replace routines), until no better solution is found after a fixed number (LocalSearch_Iteration) of consecutive operations. All 2-opt*, Move, 2-opt, Or-opt, Insert, and Replace (with Rule A and Rule B) routines aim to use local search to find a better solution than the current one in hand. Finally, we apply Remove routine a fixed number of times (Remove_Iteration) to try to get out of a potential local optimal solution. Initialization, 2-opt*, Move, 2-opt, Or-opt, Insert, Replace and Remove routines will be presented in the following subsections. The details of Procedure 2 are provided in the following.

Procedure 2 Tour planning

```

Initialization;
CurrentSolution = initial solution;
BestSolution = initial solution;
LocalSearch_Loop = 0;
Remove_Loop = 0;
While LocalSearch_Loop < LocalSearch_Iteration do
    2-opt*;
    Move;
    2-opt;
    Or-opt;
    Insert;
    If Remove_Loop < Remove_Iteration/2 then
        Replace with Rule A;
    Else
        Replace with Rule B;
    End
    Return LocalBestSolution;
    If LocalBestSolution > BestSolution then
        BestSolution = LocalBestSolution;
        LocalSearch_Loop = 0;
    Else
        LocalSearch_Loop++;
    End
    If LocalSearch_Loop == LocalSearch_Iteration & Remove_Loop < Remove_Iteration
    then
        Remove;
        Remove_Loop++;
        LocalSearch_Loop = 0;
    End
End

```

3.2 Initialization

Our initialization method is similar to the one used by Souffriau et al. (2008), but ours is deterministic rather than randomized. The initialization starts a tour with d identical routes with only hotel sites and no tourist site. For every site k not included in the tour yet, consider all its feasible insertions into the tour, i.e., those insertions with site k such that all the time and budget constraints are satisfied. We note that if site k is inserted between a pair of adjacent sites i and j , then the tour time is increased by $t_{ikj} = \tau_k + t_{ik} + t_{jk} - t_{ij}$. Among all feasible insertions, compute their marginal utility value $MU_{ikj} = U_k/t_{ikj}$, and add them to a marginal utility set Ω . The site with the maximum marginal utility value is selected and it is inserted into the current tour. The procedure is repeated until no more feasible insertion is possible, and both “CurrentSolution” and “BestSolution” are set to the initial solution.

3.3 Time reduction procedure

There are four standard routines, 2-opt*, Move, 2-opt, and Or-opt, in this procedure. These routines are based on local search ideas and have been used in various optimization problems in the literature, e.g., Potvin and Rousseau (1995), Chao et al. (1996), Lin (1965), and

Or (1976). In our case, we use them to reduce the total tour time. The purpose of reducing the total tour time is that additional sites may be inserted into the tour later in Insert and Replace routines to increase the total utility value. During the time reduction procedure we apply 2-opt*, Move, 2-opt, and Or-opt routines in sequence, and for any of these four routines, the best solution with the minimum tour time in each iteration (“LocalBestSolution”) is selected as the new “CurrentSolution.” Similar to Potvin et al. (1996), to save computation time and prevent cycling, we do not explore the entire 2-opt*, Move, 2-opt, and Or-opt neighborhoods, but only consider the q_i nearest neighbors for each selected site i , and declare those accepted solutions as Tabu for a fixed number of iterations.

3.3.1 2-opt* routine

2-opt* removes two edges in two different routes and replace them with two new edges, as depicted in Fig. 1. Suppose that site $j + 1$ is one of the q_i nearest neighbors in distance of site i and contained another route of the tour, then in 2-opt* routine, edges $(i, i + 1)$ and $(j, j + 1)$ are replaced by edges $(i, j + 1)$ and $(j, i + 1)$.

In this routine, 2-opt* neighborhood is explored until a fixed number (2opt*_Iteration) of consecutive iterations are performed without any improvement to “BestSolution”. The details of 2-opt* routine are given below.

2-opt* routine

2opt*_Loop = 0;

While 2opt*_Loop < 2opt*_Iteration **do**

 Select p sites randomly in the current tour (associated with CurrentSolution);

For Site $i = 1$ **to** p

 Use 2-opt* rule to exchange edges in the q_i -nearest-neighborhood of Site i ;

End

 LocalBestSolution = the best solution among the $p \sum_i q_i$ solutions;

 CurrentSolution = LocalBestSolution;

 The inverse move is declared tabu for t iterations (it is forbidden to insert edges that are removed this time in the following t iterations);

If the tour time of CurrentSolution is less than that of BestSolution **then**

 BestSolution = CurrentSolution;

 2opt*_Loop = 0;

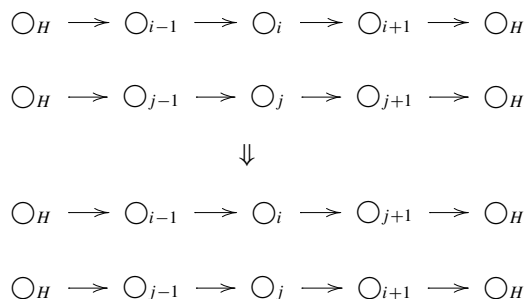
Else

 2opt*_Loop++;

End;

End

Fig. 1 2-opt* routine



3.3.2 Move, 2-opt, and Or-opt routines

In Move routine, one, two or three consecutive sites in one route is moved to a different route. For example, assuming that site i is selected and site j is one of the q_i nearest neighbors in distance of site i and contained in another route of the tour, then site i (possibly with sites $i + 1$ and $i + 2$ as well) is moved to between sites j and $j + 1$, as depicted in Fig. 2.

In 2-opt routine, 2 edges are removed from the same route and replaced by 2 new edges to create a new route. Suppose that site i is selected and site j is one of the q_i nearest neighbors in distance of site i and contained in the same route as site i , then we can replace edges $(i, i + 1)$ and $(j, j + 1)$ with edges (i, j) and $(i + 1, j + 1)$ to create a new route (see Fig. 3).

Finally, in Or-opt routine, we consider a sequence of one, two or three consecutive sites in a route, and moves them to a different location in the same route. In Fig. 4, suppose that site i is selected and site j is one of the q_i nearest neighbors in distance of site i and contained in the same route, then site i (possibly together with sites $i + 1$ and $i + 2$) is moved to between sites j and $j + 1$.

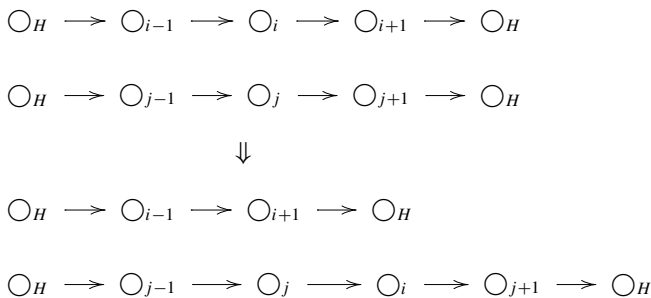


Fig. 2 Move routine

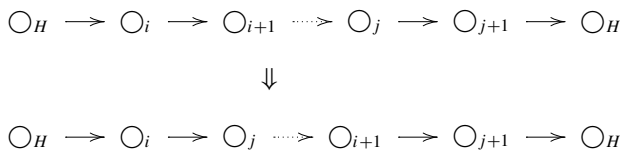


Fig. 3 2-opt routine

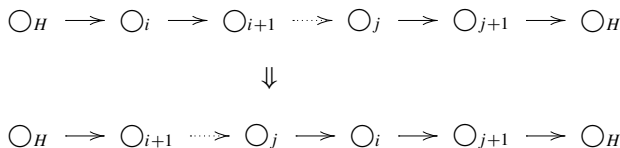


Fig. 4 Or-opt routine

The basic structures of Move, 2-opt, and Or-opt routines are almost identical to that of 2-opt* routine, except that 2-opt* edge exchange rule is replaced by their respective site and edge moving/exchange rules discussed above. Therefore, their details are omitted here.

3.4 Utility increase procedure

In the utility increase procedure, two routines, Insert and Replace, are used, aiming at increasing the total utility of the tour.

3.4.1 Insert

Insert routine works in a very similar way as Initialization routine and it tries to insert new locations into the tour to increase the total utility. As discussed in Initialization routine, for each iteration, a marginal utility set Ω containing MU_{ikj} is obtained. Then s sites with the largest marginal utility values in Ω are selected as candidates for inserting. We note that s can be set randomly. Next one member is randomly selected from the s candidates and is inserted into the tour, and this procedure is repeated until no more feasible insertion is possible. The solution obtained from Insert routine is set as “CurrentSolution” for Replace routine.

3.4.2 Replace

Replace routine attempts to replace a site included in the current tour by a non-included site. First, we select N_1 sites with the lowest utility and N_2 sites with the lowest marginal utility in the tour (note that the marginal utility of site k , which is between sites i and j in the tour, has the same value as MU_{ikj}). Let N_3 be the number of all sites selected. Since some sites may be selected in both, it is possible that $N_3 < N_1 + N_2$. Also, note that the values of N_1 and N_2 may be different for each iteration. We next order all the selected sites based on their marginal utility values (from the lowest to the highest), and remove these sites one by one from their routes. Insert routine presented in Sect. 3.4.1 is applied after each site is removed. Replace routine ends when all the selected sites are removed. In additional, we use two different rules to reset “Currentsolution:” (1) Rule A, only when the current solution is better than “CurrentSolution”, it is updated as the new “CurrentSolution”, and (2) Rule B, the next “CurrentSolution” is always set to the current solution. The details of the routine with the two different rules are provided in the following.

Replace routine with Rule A

```

For  $i = 1$  to  $N_3$ 
    Remove site  $i$ ;
    Insert;
    If the solution obtained from Insert is better than CurrentSolution then
        CurrentSolution = the solution obtained from Insert;
    End
End
LocalBestSolution = CurrentSolution.
  
```

Replace routine with Rule B

```

For  $i = 1$  to  $N_3$ 
    Remove site  $i$ ;
    Insert;
  
```

CurrentSolution = the solution obtained from Insert;

End

LocalBestSolution = the best solution among all the N_3 above solutions obtained from Insert.

3.5 Remove routine

In order to avoid sticking at local optima, we propose to use Remove routine. Let Θ be the set of all the sites contained in the current tour. We first divide the sites in Θ in two different ways: one based on their utility values and the other based on their marginal utility values, i.e., $\Theta = \Theta_{u1} + \Theta_{u2} = \Theta_{m1} + \Theta_{m2}$, where Θ_{u1} contains the sites in Θ with the highest utility values (Θ_{u2} contains the remaining sites in Θ) and Θ_{m1} contains the sites in Θ with the highest marginal utility values (Θ_{m2} contains the remaining sites in Θ). In Remove routine, the sites in $\Theta_{u2} \cap \Theta_{m2}$ are removed from the tour with probability p_1 , the sites in $\Theta_{u1} \cap \Theta_{m1}$ are removed from the tour with probability p_2 , and the remaining sites in Θ are removed from the tour with probability p_3 . In general, we set $p_1 > p_3 > p_2$. Also, we increase $p_1, p_2, p_3, |\Theta_{u2}|$, and $|\Theta_{m2}|$ as more iterations are carried out. The final solution obtained in Remove routine is set as “CurrentSolution.”

4 Numerical experiments: the tour planning problem

In this section, we provide some numerical experiments for two tour planning examples (one small and one large). Both examples are taken from our work in a real-world problem. Values of the major parameters used in the algorithm are given in Table 1, where P denotes the number of sites in the current tour or route, and Q_i denotes the size of the entire neighborhood of site i associated with the current tour.

Example 1 has 30 sites and 3 hotels, with two different utility functions. We assume that there is no constraint on the total budget as well as the time during which the tourist can visit a site. We also assume that T_k is the same for all d days, so let $T_k = T$. We use CPLEX to solve the MILP and compare the results with those obtained based on our method. Our selection rules are probabilistic, so for our method we run 10 replicates for each instance, based on which the average, and maximum objective values are then obtained. All our numerical experiments, both in this section and the next section, are run on a Pentium(R) 4 processor PC with 1.7 GHz and 512 MB Ram. The numerical results for Example 1 are presented in Table 2. Based on these results it is clear that our method works extremely well: in 15 cases the error of our method are almost zero while in other 3 cases the error is 0.85%, 2.84%, and 0.12% respectively, where the error = (Optimal value – Average value of our method)/Optimal value. The average CPU times of our method are between 0.2 and 0.6 second, which are considerably less than those of CPLEX.

Table 1 Values of the major parameters used in the algorithm

Name	Value	Name	Value
Remove_Iteration	20	Oropt_Iteration	10
LocalSearch_Iteration	5	p	$[0.5P]$
2opt*_Iteration	10	q_i	$[0.5Q_i]$
Move_Iteration	10	R	6
2opt_Iteration	10	t	5

Table 2 Results on example 1

d	T	Our method			MILP		Error (%)
		Max	Aver	Time (s)	Obj	Time (s)	
Utility function 1							
1	8	287	287	0.264	287	0.563	0.00
	10	379	379	0.279	379	2.563	0.00
	12	411	411	0.344	411	8.953	0.00
2	8	515	515	0.399	515	4.891	0.00
	10	633	633	0.478	633	134.031	0.00
	12	762	762	0.591	762	1433.328	0.00
3	8	701	701	0.483	707	29.953	0.85
4	8	888	888	0.534	888	163.547	0.00
5	8	1055	1055	0.592	1055	2596.640	0.00
Utility function 2							
1	8	177	177	0.286	177	0.875	0.00
	10	245	245	0.275	245	2.312	0.00
	12	272	272	0.353	272	10.688	0.00
2	8	340	340	0.416	340	13.344	0.00
	10	415	413.5	0.489	424	590.532	2.48
	12	535	535	0.594	535	9220.903	0.00
3	8	492	492	0.475	492	42.344	0.00
4	8	631	631	0.553	631	92.406	0.00
5	8	763	762.1	0.587	763	3740.229	0.12

Table 3 Results on example 2

d	T	Our method			Upperbound	Gap (%)
		Max	Aver	Time (s)		
Utility function 3						
1	8	281	281	0.844	305	7.87
	10	341	341	0.867	392	13.01
	12	399	399	1.358	462	13.64
2	8	469	469	2.227	526	10.84
	10	603	603	2.484	682	11.58
	12	728	727.6	3.710	801	9.16
3	8	667	667	2.388	742	10.11
	10	841	841	3.658	942	10.72
	12	1013	1013	5.359	1116	9.23
4	8	841	841	3.202	941	10.63
	10	1071	1067.1	7.435	1192	10.48
	12	1286	1278.1	7.188	1416	9.74
5	8	1011	1011	4.053	1139	12.12
	10	1271	1268	13.344	1454	12.79
	12	1555	1536.8	17.600	1706	9.92

Table 3 (Continued)

d	T	Our method			Upperbound	Gap (%)
		Max	Aver	Time (s)		
Utility function 4						
1	8	234	234	1.123	279	16.13
	10	304	304	2.900	335	9.25
	12	370	370	1.712	413	10.41
2	8	422	422	4.633	499	15.43
	10	562	562	7.149	626	10.22
	12	665	665	3.989	752	11.57
3	8	610	610	2.597	712	14.33
	10	795	787.9	5.244	898	12.26
	12	950	941.4	5.688	1058	11.02
4	8	785	785	3.405	903	13.07
	10	1002	999	10.625	1137	12.14
	12	1201	1196.1	13.437	1324	9.66
5	8	950	950	4.206	1091	12.92
	10	1200	1198.9	8.106	1341	10.60
	12	1444	1432.9	19.677	1580	9.31

Table 4 Comparison between our algorithm and Ant and GLS algorithms: average objective value

Set name	Set 4	Set 5	Set 6	Set 7	Sets 4–7
Aver gap with Ant	1.28	0.46	−0.41	1.62	0.89
Aver gap with GLS	0.07	−0.42	−0.86	−0.01	−0.23

Table 5 Comparison between our algorithm and Ant and GLS algorithms: maximum objective value

Results of our algorithm	Better	Equal	Worse
Ant	3	116	93
GLS	110	76	26

Table 6 Comparison between our algorithm and Ant and GLS algorithms: average CPU time

Set name	Set 4	Set 5	Set 6	Set 7	Sets 4–7
Our algorithm	14.6	3.8	3.1	4.3	6.7
Ant	37.1	17.4	16.1	30.4	26.0
GLS	12.7	5.0	5.8	16.7	10.3

Example 2 has 59 sites and 26 hotels and is similar to Example 1. For this example, it is no longer possible to use CPLEX to solve the MILP directly. So we relax the integer constraints in TPP and obtain upper bounds for TPP. We then compare the results obtained by our method with these upper bounds. The numerical results are presented in Table 3. It is clear that our method produces reasonably good solutions in fairly short amount of times

Table 7 Results on Set 4

Ins.	Our method			Ant			GLS	Gap with	
	Max	Aver	Time	Max	Aver	Time	Obj	Ant	GLS
4.2.a	206	201	0.8	206	206	16.2	206	2.43	2.43
4.2.b	341	333.6	1.4	341	338.7	20.3	341	1.51	2.17
4.2.c	452	450.6	2.2	452	447.9	21.9	452	−0.60	0.31
4.2.d	528	520.3	2.9	531	527.5	23.5	528	1.36	1.46
4.2.e	607	596.9	4.3	618	596.9	25.5	613	0.00	2.63
4.2.f	677	670.9	5.5	687	672.6	27.5	670	0.25	−0.13
4.2.g	744	729.7	7.6	757	736.8	29.7	733	0.96	0.45
4.2.h	803	793.3	10.1	827	818.2	31.6	791	3.04	−0.29
4.2.i	865	845.6	10.4	918	894.1	31.9	845	5.42	−0.07
4.2.j	894	889.2	12.4	965	953.2	34.6	956	6.71	6.99
4.2.k	959	945.3	14.8	1022	1001.1	36.1	945	5.57	−0.03
4.2.l	1021	1011	17.7	1071	1063.5	37.5	1040	4.94	2.79
4.2.m	1081	1071.3	22.4	1130	1110.6	39.9	1040	3.54	−3.01
4.2.n	1123	1114.4	27.5	1168	1146.9	41.3	1111	2.83	−0.31
4.2.o	1162	1153.2	33.3	1215	1175.8	40.4	1140	1.92	−1.16
4.2.p	1222	1204.8	40.9	1242	1215	43	1220	0.84	1.25
4.2.q	1249	1245.3	47.1	1263	1234.3	43.6	1253	−0.89	0.61
4.2.r	1280	1275.9	52.6	1288	1263.4	45	1281	−0.99	0.4
4.2.s	1299	1293.1	60.2	1304	1288.4	46	1286	−0.36	−0.55
4.2.t	1306	1303.8	66.4	1306	1304.4	47.1	1298	0.05	−0.45
4.3.b	38	38	0.4	38	38	17.4	38	0.00	0.00
4.3.c	193	192.4	0.8	193	193	22	193	0.31	0.31
4.3.d	335	334.2	1.4	335	333	24.9	333	−0.36	−0.36
4.3.e	468	460.6	2.1	468	463.2	28.3	468	0.56	1.58
4.3.f	579	566.1	2.8	579	569.2	30.5	553	0.54	−2.37
4.3.g	651	642.5	3.8	653	651.6	31.6	645	1.40	0.39
4.3.h	719	710.5	4.9	720	712.6	34.4	707	0.29	−0.50
4.3.i	805	779.2	6.0	796	779.2	36	768	0.00	−1.46
4.3.j	849	840.6	7.3	861	839.4	38.3	848	−0.14	0.87
4.3.k	894	889.6	8.5	918	895.7	38.5	881	0.68	−0.98
4.3.l	953	934.7	10.3	979	954.2	39.3	948	2.04	1.40
4.3.m	1009	985.7	12.8	1053	1023.1	41	1014	3.66	2.79
4.3.n	1106	1062.7	14.5	1121	1100.3	43.2	1067	3.42	0.40
4.3.o	1134	1109.4	17.6	1170	1158.1	44	1121	4.21	1.03
4.3.p	1196	1162.2	21.0	1221	1201.7	45.5	1153	3.29	−0.80
4.3.q	1233	1197.8	24.3	1252	1227.4	46.8	1236	2.41	3.09
4.3.r	1266	1257.4	29.2	1267	1255.7	47.1	1239	−0.14	−1.49
4.3.s	1295	1289.4	34.7	1293	1283.7	48.3	1261	−0.44	−2.25
4.3.t	1304	1301.3	43.8	1305	1302.3	48.8	1297	0.08	−0.33
4.4.d	38	38	0.4	38	38	22.4	38	0.00	0.00
4.4.e	183	183	0.7	183	183	27.5	183	0.00	0.00
4.4.f	324	318.8	1.3	324	324	30.2	324	1.60	1.60
4.4.g	461	453.3	2.1	461	460.1	33.5	461	1.48	1.67
4.4.h	571	556.5	2.7	571	552	35.5	555	−0.82	−0.27
4.4.i	641	633.6	3.5	657	641.6	37.5	622	1.25	−1.86
4.4.j	730	709.3	4.2	732	726.7	39.5	694	2.39	−2.20

Table 7 (Continued)

Ins.	Our method			Ant			GLS	Gap with	
	Max	Aver	Time	Max	Aver	Time	Obj	Ant	GLS
4.4.j	730	709.3	4.2	732	726.7	39.5	694	2.39	−2.20
4.4.k	820	796.9	5.6	821	814.2	40.9	815	2.12	2.22
4.4.l	878	865.1	6.2	880	868.4	42.8	848	0.38	−2.02
4.4.m	911	900.7	7.9	918	904.7	43.4	839	0.44	−7.35
4.4.n	958	948.0	8.9	961	946.3	44.4	932	−0.18	−1.72
4.4.o	1002	994.8	10.0	1036	1001.1	45.9	969	0.63	−2.66
4.4.p	1082	1058.4	11.3	1111	1074	47	1073	1.45	1.36
4.4.q	1130	1090.3	14.1	1145	1106.2	47.5	1120	1.44	2.65
4.4.r	1201	1178.7	15.4	1200	1168.7	49.2	1143	−0.86	−3.12
4.4.s	1245	1239.6	18.5	1249	1233.9	50.2	1215	−0.46	−2.02
4.4.t	1277	1264.6	20.7	1281	1268.4	51.1	1275	0.30	0.82

Table 8 Results on Set 5

Ins.	Our method			Ant			GLS	Gap with	
	Max	Aver	Time	Max	Aver	Time	Obj	Ant	GLS
5.2.b	20	20	0.3	20	20	7	20	0.00	0.00
5.2.c	50	50	0.4	50	50	7.9	50	0.00	0.00
5.2.d	80	80	0.4	80	80	9	80	0.00	0.00
5.2.e	180	180	0.5	180	180	9.6	180	0.00	0.00
5.2.f	240	240	0.6	240	240	10.1	240	0.00	0.00
5.2.g	320	320	0.7	320	320	10.8	320	0.00	0.00
5.2.h	410	396.5	0.8	410	404.5	11.6	385	1.98	−2.99
5.2.i	480	468.5	1.0	480	480	12.3	465	2.40	−0.75
5.2.j	570	565	1.2	580	580	12.9	580	2.59	2.59
5.2.k	670	669.5	1.5	670	670	13.7	665	0.07	−0.68
5.2.l	800	776	1.9	800	778	14.5	760	0.26	−2.11
5.2.m	860	848	2.5	860	859.5	15.1	835	1.34	−1.56
5.2.n	925	921.5	3.3	925	921	15.7	920	−0.05	−0.16
5.2.o	1000	985.5	3.9	1020	1011	16.3	1010	2.52	2.43
5.2.p	1150	1109.5	4.6	1150	1143.5	16.9	1080	2.97	−2.73
5.2.q	1195	1176	5.8	1195	1194	17.7	1155	1.51	−1.82
5.2.r	1260	1253.5	7.6	1260	1258.5	18.3	1245	0.40	−0.68
5.2.s	1325	1312.5	8.8	1340	1324	19.1	1295	0.87	−1.35
5.2.t	1390	1380	10.9	1400	1382	19.7	1360	0.14	−1.47
5.2.u	1450	1443.5	13.9	1460	1452.5	20.5	1445	0.62	0.10
5.2.v	1505	1493	16.3	1505	1491.5	21.1	1500	−0.10	0.47
5.2.w	1560	1560	18.8	1560	1537.5	21.7	1560	−1.46	0.00
5.2.x	1600	1595	21.7	1610	1595.5	22.3	1610	0.03	0.93
5.2.y	1635	1629.5	16.0	1645	1631.5	22.6	1620	0.12	−0.59
5.2.z	1680	1675	16.2	1680	1672.5	23	1680	−0.15	0.30
5.3.b	15	15	0.3	15	15	9.7	15	0.00	0.00
5.3.c	20	20	0.3	20	20	9.6	20	0.00	0.00
5.3.d	60	60	0.4	60	60	10.9	60	0.00	0.00

Table 8 (Continued)

Ins.	Our method			Ant			GLS	Gap with	
	Max	Aver	Time	Max	Aver	Time	Obj	Ant	GLS
5.3.e	95	95	0.4	95	95	11.7	95	0.00	0.00
5.3.f	110	110	0.5	110	110	12.5	110	0.00	0.00
5.3.g	185	185	0.5	185	185	12.8	185	0.00	0.00
5.3.h	260	260	0.7	260	260	13.9	260	0.00	0.00
5.3.i	335	333.5	0.8	335	335	14.3	335	0.45	0.45
5.3.j	470	470	0.9	470	470	15.1	470	0.00	0.00
5.3.k	495	483.5	1.0	495	495	15.6	485	2.32	0.31
5.3.l	585	582	1.2	595	590	16.3	545	1.36	−6.79
5.3.m	650	650	1.4	650	649.5	17	645	−0.08	−0.78
5.3.n	755	755	1.6	755	755	17.6	755	0.00	0.00
5.3.o	870	854.5	2.1	870	865	18.3	870	1.21	1.78
5.3.p	990	987	2.5	990	990	18.8	960	0.30	−2.81
5.3.q	1070	1057.5	2.9	1070	1061.5	19.5	1025	0.38	−3.17
5.3.r	1120	1108.5	3.6	1125	1114.5	20	1080	0.54	−2.64
5.3.s	1185	1181	4.2	1190	1187	20.7	1130	0.51	−4.51
5.3.t	1260	1255.5	4.9	1260	1251	21.2	1245	−0.36	−0.84
5.3.u	1345	1338.5	6.1	1345	1336	21.6	1330	−0.19	−0.64
5.3.v	1415	1411	7.3	1425	1402	22.1	1420	−0.64	0.63
5.3.w	1465	1452.5	8.4	1485	1458	22.7	1460	0.38	0.51
5.3.x	1540	1529.5	5.8	1540	1513.5	23.1	1525	−1.06	−0.30
5.3.y	1590	1576.5	6.4	1590	1555	23.5	1585	−1.38	0.54
5.3.z	1635	1623.5	7.3	1635	1610	23.8	1635	−0.84	0.70
5.4.c	20	20	0.3	20	20	12.6	20	0.00	0.00
5.4.d	20	20	0.3	20	20	12.2	20	0.00	0.00
5.4.e	20	20	0.3	20	20	12.4	20	0.00	0.00
5.4.f	80	80	0.4	80	80	14.4	80	0.00	0.00
5.4.g	140	140	0.5	140	140	15.3	140	0.00	0.00
5.4.h	140	140	0.5	140	140	15.7	140	0.00	0.00
5.4.i	240	240	0.7	240	240	16.3	240	0.00	0.00
5.4.j	340	340	0.8	340	340	17.1	340	0.00	0.00
5.4.k	340	340	0.8	340	340	17.8	340	0.00	0.00
5.4.l	430	429	1.0	430	429.5	18	430	0.12	0.23
5.4.m	550	550	1.1	555	554	18.9	550	0.72	0.00
5.4.n	620	620	1.3	620	620	19.2	620	0.00	0.00
5.4.o	685	669	1.5	690	690	19.9	690	3.04	3.04
5.4.p	760	760	1.6	765	758	20.4	760	−0.26	0.00
5.4.q	860	860	1.9	860	851	21.2	830	−1.06	−3.61
5.4.r	960	906	2.0	960	960	21.7	920	5.63	1.52
5.4.s	1005	1000.5	2.5	1030	1020	22.3	1005	1.91	0.45
5.4.t	1160	1139	2.8	1160	1152	22.7	1110	1.13	−2.61
5.4.u	1300	1267.5	3.3	1300	1300	23	1300	2.50	2.50
5.4.v	1320	1310.5	4.0	1320	1320	23.5	1275	0.72	−2.78
5.4.w	1380	1373	4.5	1390	1373.5	24	1375	0.04	0.15
5.4.x	1430	1421.5	5.5	1450	1443	24.4	1410	1.49	−0.82
5.4.y	1520	1517	4.0	1520	1513	24.8	1520	−0.26	0.20
5.4.z	1620	1596.5	4.7	1620	1585.5	25.2	1575	−0.69	−1.37

Table 9 Results on Set 6

Ins.	Our method			Ant			GLS	Gap with	
	Max	Aver	Time	Max	Aver	Time	Obj	Ant	GLS
6.2.d	192	192	0.6	192	189	8.8	192	−1.59	0.00
6.2.e	360	360	0.9	360	359.4	10.3	360	−0.17	0.00
6.2.f	588	579.6	1.4	588	587.4	11.9	534	1.33	−8.54
6.2.g	660	660	1.6	660	660	12.9	660	0.00	0.00
6.2.h	780	780	2.1	780	780	14	780	0.00	0.00
6.2.i	888	888	2.8	888	888	15	888	0.00	0.00
6.2.j	936	936	3.6	948	947.4	15.9	912	1.20	−2.63
6.2.k	1032	1032	4.5	1032	1032	16.9	1032	0.00	0.00
6.2.l	1116	1112.4	6.2	1116	1111.2	17.9	1104	−0.11	−0.76
6.2.m	1188	1184.4	8.8	1188	1184.4	19.1	1158	0.00	−2.28
6.2.n	1260	1251.6	12.6	1260	1230.6	19.6	1230	−1.71	−1.76
6.3.g	282	282	0.8	282	278.4	12.6	276	−1.29	−2.17
6.3.h	444	443.4	1.2	444	427.8	13.5	438	−3.65	−1.23
6.3.i	642	635.4	1.8	642	640.8	15.3	642	0.84	1.03
6.3.j	828	828	2.2	828	825.6	16.4	828	−0.29	0.00
6.3.k	894	887.4	2.5	894	888.6	17.5	888	0.14	0.07
6.3.l	1002	994.8	3.1	1002	996	18.5	996	0.12	0.12
6.3.m	1080	1079.4	3.8	1080	1071.6	19.4	1074	−0.73	−0.50
6.3.n	1170	1165.8	4.8	1170	1159.2	20.3	1122	−0.57	−3.90
6.4.j	366	360.6	0.9	366	363	16.1	360	0.66	−0.17
6.4.k	522	518.8	1.3	528	525	16.9	528	1.18	1.74
6.4.l	696	690.6	2.1	696	671.4	18.1	684	−2.86	−0.96
6.4.m	912	900.6	2.4	912	885.6	19.5	912	−1.69	1.25
6.4.n	1068	1068	2.9	1068	1061.4	20.3	1068	−0.62	0.00

(in fact, all within 20 seconds), and the average gap between our results and their respective upper bounds is about 11.34%.

5 Numerical experiments: the team orienteering problem

The team orienteering problem is a special case of the tour planning problem in which the visiting time to each site is zero and both the starting and end points of the tour are fixed and given. The heuristic algorithms proposed by Ke et al. (2008) and Vansteenwegen et al. (2009) are among the best ones (see numerical results provided in the two papers). In this section, we apply our algorithm to data sets 4–7 of the team orienteering problem (with 100, 66, 64, and 102 vertices, respectively, and each set contains 2, 3, and 4 routes) in Chao et al. (1996). We compare our results with those from Ke et al. (2008) and Vansteenwegen et al. (2009). The values of the major parameters used in our algorithm remain the same as in Sect. 4, except that the values of Move_Iteration, Oropt_Iteration, p , and q_i are reset to 5, 5, $[0.2P]$, and $[0.2Q_i]$ respectively. For each instance, similar to Ke et al. (2008), we run 10 independent replicates, and the average and maximum objective values are provided. But

Table 10 Results on Set 7

Ins.	Our method			Ant			GLS	Gap with	
	Max	Aver	Time	Max	Aver	Time	Obj	Ant	GLS
7.2.a	30	30	0.3	30	30	12.6	30	0.00	0.00
7.2.b	64	64	0.4	64	64	13.6	64	0.00	0.00
7.2.c	101	101	0.5	101	101	14.9	101	0.00	0.00
7.2.d	190	190	0.7	190	190	16.8	190	0.00	0.00
7.2.e	290	290	1.0	290	290	18.7	279	0.00	−3.94
7.2.f	387	383.9	1.5	387	386.7	20.5	340	0.72	−12.91
7.2.g	459	456	2.1	459	459	21.8	440	0.65	−3.64
7.2.h	521	520.3	2.7	521	521	23.1	521	0.13	0.13
7.2.i	579	578.1	3.2	580	578.6	24.5	579	0.09	0.16
7.2.j	637	631.3	3.9	646	644	26.1	633	1.97	0.27
7.2.k	698	689.3	4.4	705	701.2	26.9	697	1.70	1.10
7.2.l	757	738.3	5.3	767	765.4	27.8	758	3.54	2.60
7.2.m	811	788.3	6.5	827	827	29.5	821	4.68	3.98
7.2.n	872	854.8	7.4	888	878	30.6	873	2.64	2.08
7.2.o	922	900.8	8.7	945	940.1	32.7	937	4.18	3.86
7.2.p	981	956.6	10.7	1002	991.3	33.9	984	3.50	2.78
7.2.q	1034	1003	12.2	1043	1040	35.7	1037	3.56	3.28
7.2.r	1076	1057.5	14.7	1094	1078.9	36.5	1076	1.98	1.72
7.2.s	1119	1100.9	17.9	1136	1115.2	36.9	1108	1.28	0.64
7.2.t	1154	1136.5	21.2	1179	1146.6	36.4	1155	0.88	1.60
7.3.b	46	46	0.4	46	46	18.5	46	0.00	0.00
7.3.c	79	79	0.4	79	79	19.3	79	0.00	0.00
7.3.d	117	117	0.5	117	117	20.8	117	0.00	0.00
7.3.e	175	175	0.6	175	175	22.4	175	0.00	0.00
7.3.f	247	246.3	0.9	247	247	24.1	247	0.28	0.28
7.3.g	335	334	1.2	344	344	25.6	344	2.91	2.91
7.3.h	425	420.4	1.6	425	424.3	27.7	418	0.92	−0.57
7.3.i	485	480.2	2.1	487	485.3	29.3	480	1.05	−0.04
7.3.j	557	549.2	2.6	564	563.2	30.7	539	2.49	−1.89
7.3.k	618	612.3	3.1	633	629.5	31.5	586	2.73	−4.49
7.3.l	681	676.3	3.8	684	680.7	33.1	675	0.65	−0.19
7.3.m	755	744.4	4.0	762	759.1	33.8	745	1.94	0.08
7.3.n	792	787.2	4.6	820	813.9	35.1	789	3.28	0.23
7.3.o	848	839.1	5.4	874	874	35.7	833	3.99	−0.73
7.3.p	907	899.6	6.4	929	925.6	36.8	915	2.81	1.68
7.3.q	962	950.8	7.1	987	984.5	38	957	3.42	0.65
7.3.r	1007	979.1	8.3	1026	1018.4	39.6	1013	3.86	3.35
7.3.s	1054	1036.7	9.8	1081	1070.3	41.3	1066	3.14	2.75
7.3.t	1111	1095	10.6	1118	1107.2	42.3	1102	1.10	0.64
7.4.b	30	30	0.3	30	30	23	30	0.00	0.00
7.4.c	46	46	0.4	46	46	23.6	46	0.00	0.00
7.4.d	79	79	0.4	79	79	24.5	79	0.00	0.00

Table 10 (Continued)

Ins.	Our method			Ant			GLS	Gap with	
	Max	Aver	Time	Max	Aver	Time	Obj	Ant	GLS
7.4.e	123	123	0.5	123	123	26.1	123	0.00	0.00
7.4.f	164	164	0.7	164	164	27.9	164	0.00	0.00
7.4.g	217	214.3	0.8	217	217	29.1	209	1.24	−2.54
7.4.h	285	283.4	1.0	285	285	30.7	285	0.56	0.56
7.4.i	366	353.1	1.3	366	366	32.1	359	3.52	1.64
7.4.j	462	456.8	1.7	462	462	33.9	462	1.13	1.13
7.4.k	520	513.5	2.2	520	518	35.3	499	0.87	−2.91
7.4.l	577	572.1	2.6	590	581.7	37	573	1.65	0.16
7.4.m	636	629.9	3.0	646	643.9	37.9	638	2.17	1.27
7.4.n	723	708.4	3.7	730	725.6	39.4	698	2.37	−1.49
7.4.o	775	758.2	4.0	781	777.5	40.2	739	2.48	−2.60
7.4.p	836	817.8	4.7	846	839.4	41.3	813	2.57	−0.59
7.4.q	886	876.2	5.4	909	905.1	42.1	888	3.19	1.33
7.4.r	959	952	5.8	970	969.2	42.5	938	1.77	−1.49
7.4.s	1014	999.8	6.5	1022	1017.7	44	994	1.76	−0.58
7.4.t	1062	1047.4	7.6	1077	1072.8	44.7	1020	2.37	−2.69

Vansteenwegen et al. (2009) only had one run for each instance, since there is no randomness in their algorithm. The results are presented in Tables 4–10. In all the cases, the gap ($=$ (average objective value obtained by Ant or GLS – average objective value obtained by our method)/average objective value obtained by Ant or GLS) is given as percentage and the CPU time is given in seconds, where ‘Ant’ and ‘GLS’ represent the algorithms proposed by Ke et al. (2008) and Vansteenwegen et al. (2009), respectively. Table 4 summarizes the average gap between our results and the results from Ke et al. (2008) and Vansteenwegen et al. (2009). Table 5 provides the number of instances that our algorithm performs better than, equal to, or worse than the other two algorithms, respectively. Table 6 provides the average CPU times. Tables 7–10 include the detailed results from our algorithm and the other two algorithms.

The results by Ke et al. (2008) are better than our algorithm, on average 0.89%. However, the CPU time of our algorithm is much less: the average CPU time of our algorithm is only 6.7 seconds while theirs is 26.0 seconds (in fact, they ran their algorithm on a faster PC with 3.0 GHz Intel CPU). Comparing with the GLS algorithm in Vansteenwegen et al. (2009), our algorithm performs slightly better, on average 0.23%. The average CPU time of the GLS algorithm is 10.3 seconds which is longer than ours (their experiments were also run on a faster PC Intel Pentium 4 with 2.80 GHz).

6 Conclusion

In this paper, we studied the tour planning problem. We formulated it as a mixed-integer linear programming problem, and proposed a heuristic method based on the idea of local search. We provided some numerical experiments from both the tour planning problem and the team orienteering problem, which demonstrate that the method is quite efficient and is able to find good approximate solutions in very short time.

References

- Archetti, C., Hertz, A., & Speranza, M. (2007). Metaheuristics for the team orienteering problem. *Journal of Heuristics*, 13, 49–76.
- Buhalis, D. (2003). *eTourism: information technology for strategic tourism management*. London: Prentice Hall.
- Buhalis, D., & Law, R. (2008). Twenty years on and 10 years after the internet: the state of eTourism research. *Tourism Management*, 29(4), 609–623.
- Chao, I., Golden, B., & Wasil, E. (1996). The team orienteering problem. *European Journal of Operational Research*, 88, 464–474.
- Fesenmainer, D. R., Ricci, F., Schaumlechner, E., Wober, K., & Zanella, C. (2003). DIETORECS: travel advisory for multiple decision styles. In *Information and communication technologies in tourism* (pp. 232–241). New York: Springer.
- Hannes, W. (2003). Intelligent system in travel and tourism. In *Proceedings of international joint conference on artificial intelligence 2003*.
- Ke, L., Archetti, C., & Feng, Z. (2008). Ants can solve the team orienteering problem. *Computers & Industrial Engineering*, 54, 648–665.
- Lin, S. (1965). Computer solutions of the traveling salesman problem. *Bell System Technical Journal*, 44, 2245–2269.
- Or, I. (1976). *Traveling salesman-type combinatorial problems and their relation to the logistics of blood banking*. Ph.D. thesis, Department of industrial engineering and management science, Northwest University, Evanston, IL.
- Potvin, J. Y., Kervahut, T., & Rousseau, J. M. (1996). The vehicle routing problem with time windows part I: tabu search. *INFORMS Journal on Computing*, 8(2), 158–164.
- Potvin, J. Y., & Rousseau, J. M. (1995). An exchange heuristic for routing problems with time windows. *Journal of Operational Research Society*, 46, 1433–1446.
- Ricci, F. (2002). Travel recommender systems. *IEEE Intelligent Systems*, 17(6), 55–57.
- Ricci, F., & Wether, H. (2002). Case base querying for travel planning recommendation. *Information Technology & Tourism*, 4(3–4), 215–226.
- Schafer, J. B., Konstan, J., & Riedi, J. (1999). Recommender Systems in e-commerce. In *Proceedings of the 1st acm conference on Electronic commerce 1999*.
- Souffriau, W., Vansteenwegen, P., Vanden Berghe, G., & Van Oudheusden, D. (2008). A greedy randomized adaptive search procedure for the team orienteering problem. In *Proceedings of EU/Meeting 2008, France, 23–24 October 2008*.
- Vansteenwegen, P., Souffriau, W., Vanden Berghe, G., & Van Oudheusden, D. (2009). A guided local search metaheuristic for the team orienteering problem. *European Journal of Operational Research*, 196(1), 118–127.